

Variational Autoencoder

with Reparameterization Trick on MNIST

Deep Learning Laboratory Report

Topic: Probabilistic Generative Models
Dataset: MNIST Handwritten Digits
Framework: PyTorch
Date: April 1, 2026

Introduction

A **Variational Autoencoder (VAE)** is a generative model that combines ideas from Bayesian inference and deep learning. Unlike a standard (deterministic) autoencoder, a VAE learns a *probability distribution* over the latent space rather than a fixed point encoding. This makes it possible to sample from the learned distribution and generate entirely new, plausible data points—a capability that ordinary autoencoders fundamentally lack.

This report documents the design, implementation, and evaluation of a VAE trained on the MNIST handwritten-digit dataset. The key mathematical device that makes the VAE trainable via gradient descent is the **reparameterization trick**, which is explained in detail below together with how it appears concretely in the provided code.

Background: Standard Autoencoder vs. Variational Autoencoder

Standard Autoencoder

A standard autoencoder consists of two deterministic neural networks:

- **Encoder** $f_\phi : \mathcal{X} \rightarrow \mathcal{Z}$ — maps input $\mathbf{x}$ to a fixed latent code $\mathbf{z}$.
- **Decoder** $g_\theta : \mathcal{Z} \rightarrow \mathcal{X}$ — reconstructs $\hat{\mathbf{x}}$ from $\mathbf{z}$.

Training minimises only the *reconstruction loss* (e.g. MSE or BCE). The latent space is completely unstructured: similar inputs may map to distant points, and randomly sampled points $\mathbf{z}$ almost never decode into meaningful outputs. The bottleneck layer outputs a single deterministic vector with no notion of uncertainty.

Variational Autoencoder

A VAE treats the latent vector $\mathbf{z}$ as a *random variable*. The encoder outputs the *parameters* of a distribution rather than a fixed code:

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2)) \quad (1)$$

where $\boldsymbol{\mu}$ and $\log \boldsymbol{\sigma}^2$ are produced by two separate linear heads on top of the shared encoder backbone. The decoder then receives a *sample* $\mathbf{z} \sim q_\phi(\mathbf{z} \mid \mathbf{x})$ rather than a deterministic code. Training maximises the **Evidence Lower Bound (ELBO)**:

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z} \mid \mathbf{x})} [\log p_\theta(\mathbf{x} \mid \mathbf{z})]}_{\text{Reconstruction term}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z}))}_{\text{KL regularisation term}} \quad (2)$$

The KL divergence term regularises the latent space to be close to a standard Gaussian prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, ensuring continuity and enabling smooth interpolation.

Key Architectural Differences

Property	Standard AE	Variational AE
Bottleneck output	Single vector $\mathbf{z}$	$\boldsymbol{\mu}$ and $\log \boldsymbol{\sigma}^2$
Sampling	None (deterministic)	$\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$
Training objective	Reconstruction loss only	ELBO = Recon loss – KLD
Latent space structure	Unstructured	Approximately Gaussian
Generative capability	None	Yes (sample from prior)
Gradient flow	Direct	Via reparameterization trick

The Reparameterization Trick

The Problem

To train a VAE with gradient descent, we need to back-propagate through the sampling operation $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$. Sampling is inherently stochastic and has *no deterministic gradient* with respect to $\boldsymbol{\mu}$ or $\boldsymbol{\sigma}$.

The Solution

The reparameterization trick decouples the stochasticity from the learnable parameters by re-writing the sample as:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (3)$$

Now $\boldsymbol{\varepsilon}$ is sampled from a fixed, parameter-free distribution. The parameters $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ appear only in a *deterministic* linear transformation of $\boldsymbol{\varepsilon}$, so gradients can flow through them via ordinary back-propagation.

In the code, $\log \boldsymbol{\sigma}^2$ (denoted `log_var`) is predicted instead of $\boldsymbol{\sigma}$ directly, for numerical stability:

$$\boldsymbol{\sigma} = \exp\left(\frac{1}{2} \log_var\right) \implies \mathbf{z} = \boldsymbol{\mu} + \exp\left(\frac{1}{2} \log_var\right) \odot \boldsymbol{\varepsilon} \quad (4)$$

Implementation in the Code (Deliverable Cell)

Below is the exact implementation from the notebook (Cell 7 — the assessed deliverable). The figure placeholder on the following page is reserved for a screenshot of this cell as it appears in Jupyter.

Listing 1: Reparameterization Layer and Function (VAE_MNIST.ipynb – Cell 7)

```

1 class ReparameterizationLayer(nn.Module):
2     """
3     Reparameterization layer for VAE bottleneck.
4     Takes encoder output and samples from latent
5     distribution N(mu, sigma^2).
6     """
7     def __init__(self, latent_dim):
8         super(ReparameterizationLayer, self).__init__()

```

```

9         self.latent_dim = latent_dim
10
11     def forward(self, mu, log_var):
12         std = torch.exp(0.5 * log_var)      # sigma = exp(0.5 *
log_var)
13         eps = torch.randn_like(std)         # epsilon ~ N(0, I)
14         z = mu + eps * std                  # z = mu + sigma *
epsilon
15         return z
16
17 # Standalone convenience function
18 def reparameterize(mu, log_var):
19     std = torch.exp(0.5 * log_var)
20     eps = torch.randn_like(std)
21     z = mu + eps * std
22     return z

```

VAE Architecture and Loss Function

Network Architecture

The VAE is a fully-connected network operating on flattened $28 \times 28 = 784$ -dimensional MNIST images. The architecture consists of:

Encoder: $784 \xrightarrow{\text{ReLU}} 512 \xrightarrow{\text{ReLU}} 256$

Bottleneck: Two parallel linear layers $256 \rightarrow 20$ producing $\boldsymbol{\mu}$ and $\log \boldsymbol{\sigma}^2$

Reparameterization: $\mathbf{z} = \boldsymbol{\mu} + \exp(\frac{1}{2} \log \boldsymbol{\sigma}^2) \odot \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

Decoder: $20 \xrightarrow{\text{ReLU}} 256 \xrightarrow{\text{ReLU}} 512 \xrightarrow{\text{Sigmoid}} 784$

The Sigmoid activation in the final decoder layer ensures outputs lie in $[0, 1]$, matching the normalised pixel values of MNIST.

Loss Function

The total VAE loss is the negative ELBO:

$$\mathcal{L}_{\text{total}} = \underbrace{\mathcal{L}_{\text{BCE}}(\hat{\mathbf{x}}, \mathbf{x})}_{\text{Reconstruction}} + \underbrace{\left(-\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2) \right)}_{\text{KL Divergence}} \quad (5)$$

Listing 2: VAE Loss Function

```

1 def vae_loss(recon_x, x, mu, log_var):
2     bce_loss = nn.BCELoss(reduction='sum')
3     recon_loss = bce_loss(recon_x, x)
4     kld_loss = -0.5 * torch.sum(
5         1 + log_var - mu.pow(2) - log_var.exp())
6     return recon_loss + kld_loss, recon_loss, kld_loss

```

Training Configuration

- **Optimiser:** Adam ($\text{lr} = 10^{-3}$)
- **Epochs:** 10
- **Batch size:** 128
- **Latent dimension:** 20
- **Dataset:** MNIST — 60,000 training / 10,000 test images

Results and Reconstruction Quality

After 10 epochs of training, the model is evaluated on the test set by passing 10 unseen images through the full encoder–reparameterization–decoder pipeline and comparing the originals with their reconstructions.

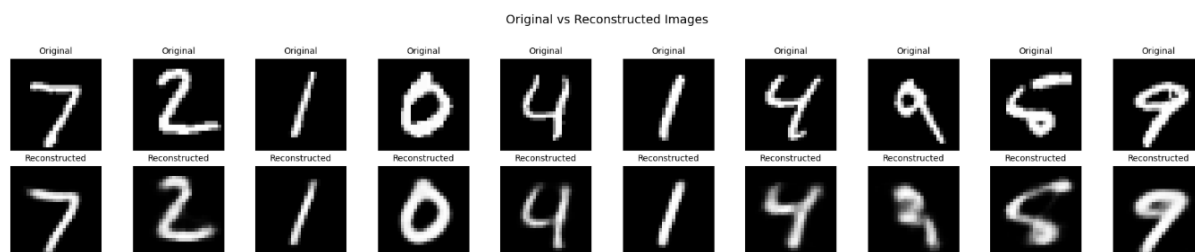


Figure 1: Reconstructed images from the MNIST using the VAE.

The reconstructed images are expected to be slightly blurrier than the originals because the VAE decoder must synthesise outputs from a *sampled* (not exact) latent code. This blurriness is a direct consequence of averaging over the posterior distribution — it is a known characteristic of VAEs and a trade-off for obtaining a smooth, continuous latent space that supports generation.

Discussion

Why the Reparameterization Trick Is Essential

Without this trick, the gradient of the loss with respect to μ and σ would need to pass through a stochastic node — an operation undefined for standard automatic differentiation. Score-function estimators (REINFORCE) exist as an alternative but have extremely high variance. The reparameterization trick provides a low-variance, unbiased gradient estimate at no additional computational cost, making VAE training practical and stable.

Latent Space Continuity

The KL divergence term penalises encoders that map inputs to overly narrow or displaced Gaussians. This forces the posterior $q_\phi(\mathbf{z}|\mathbf{x})$ to remain close to the prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$, resulting in a latent space where nearby points decode into semantically similar images. This is the property that enables smooth interpolation and controlled generation — both impossible in a standard autoencoder.

VAE vs. Standard Autoencoder — Summary

A standard autoencoder encodes $\mathbf{x}$ to a single point $\mathbf{z}$: expressive but unstructured. A VAE encodes $\mathbf{x}$ to a *region* (a Gaussian) in latent space, balancing expressiveness against regularisation. The reparameterization trick is the bridge that allows this probabilistic formulation to be trained end-to-end with gradient descent, giving the VAE its generative power while retaining the reconstruction accuracy of the autoencoder.

Conclusion

This report presented the complete design and implementation of a Variational Autoencoder for MNIST digit reconstruction. The reparameterization trick — implemented via $\mathbf{z} = \mu + \exp(0.5 * \log_var) * \text{eps}$, where $\text{eps} \sim \mathcal{N}(0, \mathbf{I})$ — is the cornerstone of the approach: it converts an intractable stochastic gradient problem into a straightforward deterministic one. The combined BCE + KLD loss function trains the model to simultaneously reconstruct inputs faithfully and maintain a well-structured Gaussian latent space. This architecture is fundamentally superior to a standard autoencoder for generative tasks, offering smooth latent interpolation, controllable sampling, and principled uncertainty estimation.